

# Batching Hamiltonians with NumPy

A practical study note for fast parameter sweeps, band structures, and susceptibility calculations

|                       |                                                                                    |
|-----------------------|------------------------------------------------------------------------------------|
| <b>Core idea</b>      | Build many small Hamiltonians as one higher-dimensional array and diagonalize them |
| <b>Typical shape</b>  | (N_points, N_orb, N_orb)                                                           |
| <b>Key NumPy tool</b> | Broadcasting with [:, None, None]                                                  |
| <b>Main benefit</b>   | Remove slow Python loops and reduce repeated Python-to-LAPACK calls.               |

## The mental model to remember

If a Hamiltonian can be written as a sum of fixed matrices multiplied by parameter-dependent scalar coefficients, batching is usually straightforward.

Generic form:

$$H(p) = c1(p) M1 + c2(p) M2 + c3(p) M3 + \dots$$

For many values  $p[0]$ ,  $p[1]$ , ...,  $p[N-1]$ , make the coefficients arrays of shape  $(N,)$ , reshape them to  $(N,1,1)$ , and multiply by fixed matrices of shape  $(1,d,d)$ . NumPy then broadcasts the result to  $(N,d,d)$ .

Prepared as a reusable reference for future numerical-physics codes.

# 1. Why batching matters

The slow pattern is common in scientific Python: loop over momentum or field values in Python, build one small matrix, diagonalize it, and repeat. Even when each diagonalization is fast, the repeated Python overhead can dominate.

```
for i in range(Nk):
    H = Hamiltonian(kx[i], ky[i])
    energies[i] = np.linalg.eigvalsh(H)
```

This performs  $N_k$  separate Python iterations and  $N_k$  separate high-level eigensolver calls.

The batched version instead constructs a stack of Hamiltonians:

```
H_all = Hamiltonian_batch(kx, ky) # shape: (Nk, 8, 8)
energies = np.linalg.eigvalsh(H_all) # shape: (Nk, 8)
```

The numerical linear algebra still has to solve all the eigenproblems. The speedup comes because Python is no longer individually orchestrating every tiny calculation.

A useful rule of thumb: NumPy is strongest when you ask it to do a large amount of homogeneous work in one operation, rather than millions of tiny operations inside Python loops.

## 2. The shape picture

For one 8-band Hamiltonian:

```
H.shape = (8, 8)
```

For  $N_k$  momentum points, batching stores all Hamiltonians in one 3D array:

```
H_batch.shape = (N_k, 8, 8)
```

| Axis | Meaning                        |
|------|--------------------------------|
| 0    | which momentum/parameter point |
| 1    | matrix row / basis index       |
| 2    | matrix column / basis index    |

Thus `H_batch[17]` is just the ordinary 8x8 Hamiltonian at parameter point 17.

### 3. Broadcasting: the central trick

Suppose a Hamiltonian contains a term

$$H(k) \text{ contains } \sin(a k_x) * M$$

where M is a fixed 8x8 matrix. For one scalar  $k_x$  this is simple: scalar times matrix.

```
term = np.sin(a * kx) * M
```

For many  $k_x$  values,  $\text{np.sin}(a*k_x)$  has shape  $(N_k,)$ , while M has shape (8,8). We add singleton dimensions so NumPy knows that each coefficient should multiply a full matrix:

```
coeff = np.sin(a * kx)           # (Nk,)
coeff3 = coeff[:, None, None]    # (Nk, 1, 1)
M3 = M[None, :, :]              # (1, 8, 8)
term = coeff3 * M3               # (Nk, 8, 8)
```

$$(N_k, 1, 1) \times (1, 8, 8) \rightarrow (N_k, 8, 8)$$

The dimensions of size 1 are broadcast: NumPy conceptually repeats the scalar coefficient across every matrix element, and repeats the fixed matrix across all parameter points.

### 4. A tiny 2x2 example

```
M = np.array([[1, 2],
              [3, 4]])

c = np.array([10, 20, 30])

result = c[:, None, None] * M[None, :, :]
print(result.shape)  # (3, 2, 2)
```

Conceptually the result is three separate matrices:

```
result[0] = 10*M = [[10, 20],
                   [30, 40]]

result[1] = 20*M = [[20, 40],
                   [60, 80]]

result[2] = 30*M = [[30, 60],
                   [90, 120]]
```

No Python loop is needed, but the interpretation is exactly the same as looping over the three coefficients.

## 5. Turning a scalar Hamiltonian into a batched Hamiltonian

Suppose the model is

$$H(k) = f_1(k) A + f_2(k) B + f_3(k) C$$

A scalar implementation is:

```
def Hamiltonian(k):  
    return f1(k)*A + f2(k)*B + f3(k)*C
```

The batched implementation is:

```
def Hamiltonian_batch(k):  
    c1 = f1(k) # shape (Nk,)   
    c2 = f2(k)   
    c3 = f3(k)   
  
    H = (  
        c1[:, None, None] * A[None, :, :]   
        + c2[:, None, None] * B[None, :, :]   
        + c3[:, None, None] * C[None, :, :]   
    )  
    return H
```

If `k.shape == (10000,)`, then `H.shape == (10000,N,N)`.

## 6. Separate constant matrices from parameter dependence

This is often the most important restructuring step. Many physics Hamiltonians consist of fixed basis matrices multiplied by functions of momentum, magnetic field, electric field, strain, etc.

### Bad pattern:

```
def Hamiltonian(kx, ky):  
    # repeated every call  
    p0, p1, p2, p3 = PauliMatrices()  
    g1 = ...  
    g2 = ...  
    M1 = ...  
    M2 = ...  
  
    # only this part actually depends on k  
    return np.sin(kx)*M1 + np.cos(ky)*M2
```

### Better pattern:

```
# Construct once  
p0, p1, p2, p3 = PauliMatrices()  
g1 = ...  
g2 = ...  
M1 = ...  
M2 = ...  
  
def Hamiltonian_batch(kx, ky):  
    c1 = np.sin(kx)  
    c2 = np.cos(ky)  
    return (
```

```

        c1[:, None, None] * M1[None, :, :]
        + c2[:, None, None] * M2[None, :, :]
    )

```

Think of the Hamiltonian as: parameter-dependent coefficients + parameter-independent matrices.

## 7. Example pattern for a k-dependent tight-binding Hamiltonian

A typical term may contain trigonometric or phase factors such as:

```

sin_ax = np.sin(a * kx)          # (Nk,)
cos_ax = np.cos(a * kx)
cos_by = np.cos(b * ky)
exp_ax = np.exp(1j * a * kx)
exp_mby = np.exp(-1j * b * ky)
exp_d1 = np.exp(1j * d1)

```

Then a term such as

$$tdab * \exp(-i b ky) * (1 + \exp(i a kx)) * \exp(i d1) * M$$

is batched as:

```

coefficient = (
    tdab
    * exp_mby
    * (1.0 + exp_ax)
    * exp_d1
)

```

```
H += coefficient[:, None, None] * M[None, :, :]
```

All coefficient arrays have shape (N<sub>k</sub>,), so every Hamiltonian term follows the same broadcasting pattern.

## 8. Batched Hermitian conjugation

For a single matrix, `H.conj().T` gives the Hermitian conjugate. For a batch `H.shape = (Nk,N,N)`, plain `H.T` reverses all axes and is not what you want.

Use either:

```
H_hc = H.conj().transpose(0, 2, 1)
```

or the very readable form:

```
H_hc = H.swapaxes(-1, -2).conj()
```

Both apply the Hermitian conjugate independently to each matrix in the batch.

## 9. Batched diagonalization

Once the Hamiltonian stack has shape (N<sub>k</sub>,N,N), NumPy can diagonalize the complete stack in one high-level call:

```

H = Hamiltonian_batch(kx, ky)      # (Nk, 8, 8)
energies = np.linalg.eigvalsh(H)   # (Nk, 8)

```

For a Hermitian Hamiltonian, use `eigvalsh` when only eigenvalues are needed. If eigenvectors are also required, use `eigh`.

## 10. Combining Hamiltonian batching with a Lindhard sum

For each fixed  $q$ , all  $k+q$  points can be processed simultaneously:

```
kx_q = kx + qx_value
ky_q = ky + qy_value

H_kq = Hamiltonian_batch(kx_q/a, ky_q/b)
energies_kq = np.linalg.eigvalsh(H_kq) - mu
occupations_kq = fermi_function(energies_kq)
```

Then use another broadcasting step for all band pairs:

```
numerator = (
    occupations_kq[:, :, None]
    - occupations_k[:, None, :]
)

denominator = (
    omega
    + energies_kq[:, :, None]
    - energies_k[:, None, :]
    + 1j*eta
)

chi_q = -np.sum(numerator / denominator) / num_k_points
```

Here the shape  $(N_k, 8, 8)$  has a different physical meaning than the Hamiltonian batch:

| Axis | Meaning in the Lindhard array |
|------|-------------------------------|
| 0    | momentum $k$                  |
| 1    | band $i$ at $k+q$             |
| 2    | band $j$ at $k$               |

Therefore `np.sum(...)` performs the sums over  $k$ ,  $i$ , and  $j$  in compiled NumPy operations.

## 11. What None / np.newaxis really does

`None` and `np.newaxis` are identical in indexing. They insert a dimension of size 1 without copying the data.

```
x.shape           # (8,)
x[:, None].shape  # (8,1)
x[None, :].shape  # (1,8)
x[:, None, None].shape # (8,1,1)
```

The purpose is usually not the singleton dimension itself; the purpose is to make arrays compatible for broadcasting.

## 12. Batching more than one parameter

The same idea extends naturally to multiple parameters. Suppose

$$H(k,E) = H_0 + f(k) H_k + E H_E$$

with  $N_k$  momentum points and  $N_E$  electric-field values. One possible batch shape is  $(N_E, N_k, N, N)$ :

```
H = (  
    H0[None, None, :, :]  
    + f(k)[None, :, None, None] * Hk[None, None, :, :]  
    + E[:, None, None, None] * HE[None, None, :, :]  
)  
  
# H.shape -> (NE, Nk, N, N)  
energies = np.linalg.eigvalsh(H)  
# energies.shape -> (NE, Nk, N)
```

This is powerful for parameter sweeps, but memory grows rapidly with every extra batch dimension.

## 13. Memory estimation

Before batching a very large problem, estimate memory. For complex128, each element uses 16 bytes. A Hamiltonian batch of shape  $(N_k, N, N)$  requires approximately:

$$\text{memory} = N_k * N * N * 16 \text{ bytes}$$

Example:  $N_k = 90,601$  and  $N = 8$ :

$$90,601 * 8 * 8 * 16 \text{ bytes} \approx 93 \text{ MB}$$

But remember that temporary arrays can multiply the peak memory. A complicated Hamiltonian may create several intermediate arrays with the same shape.

If memory becomes an issue, batch in chunks. For example, process 5,000 or 20,000  $k$  points at a time. Chunking preserves most of the speed benefit while limiting peak RAM.

## 14. When batching helps - and when it does not

| Good candidate                                   | Less suitable                                                   |
|--------------------------------------------------|-----------------------------------------------------------------|
| Many small matrices with the same size           | Very large matrices where each solve already dominates          |
| Parameter dependence through scalar coefficients | Highly irregular control flow for every parameter point         |
| Band-structure sweeps                            | Cases where only a few parameter points are needed              |
| Magnetic/electric-field sweeps                   | Memory-limited problems with huge matrix dimensions             |
| Small tight-binding or k.p Hamiltonians          | Sparse giant Hamiltonians better handled by sparse eigensolvers |

## 15. A reusable checklist for future codes

Step 1 - Identify the slow Python loop. Are you building or diagonalizing many matrices one at a time?

Step 2 - Separate constant and variable pieces. Move every parameter-independent matrix outside the loop/function.

Step 3 - Vectorize scalar coefficients. Make  $\sin(k)$ ,  $\cos(k)$ ,  $\exp(ik)$ , field values, etc. into arrays of shape  $(N,)$ .

Step 4 - Add matrix axes. Use `coeff[:,None,None]` and `M[None,:,:]`.

Step 5 - Assemble the batch. The result should have shape  $(N, \text{Norb}, \text{Norb})$ .

Step 6 - Use batched linear algebra. For Hermitian eigenvalues, `np.linalg.eigvalsh(H_batch)`.

Step 7 - Verify numerically. Compare a few batch matrices/eigenvalues against the old scalar implementation.

Step 8 - Only then parallelize the outer independent parameter. For a susceptibility calculation,  $q$  is often the natural multiprocessing axis.

### The one shape identity worth memorizing

$$(N, 1, 1) \times (1, d, d) \rightarrow (N, d, d)$$

If that pattern becomes intuitive, many vectorization opportunities in numerical physics become obvious.

## 16. Debugging and verification tips

```
# Compare old scalar and new batch Hamiltonians
kx_test = np.array([0.1, 0.2, -0.3])
ky_test = np.array([0.05, -0.1, 0.4])
```

```
H_batch = Hamiltonian_batch(kx_test, ky_test)
```

```
for i in range(len(kx_test)):
    H_old = Hamiltonian(kx_test[i], ky_test[i])
    print(np.max(np.abs(H_batch[i] - H_old)))
```

Differences near  $1e-14$  to  $1e-15$  are typically just floating-point roundoff for double-precision calculations.

```
# Hermiticity check
err = np.max(
    np.abs(H_batch - H_batch.swapaxes(-1, -2).conj())
)
print("Hermiticity error =", err)
```

Also compare eigenvalues from both implementations before deleting the old reference code.



## Quick reference sheet

| Goal                                       | Pattern                                           |
|--------------------------------------------|---------------------------------------------------|
| Add batch axis to coefficient              | <code>c[:, None, None]</code>                     |
| Add batch axis to matrix                   | <code>M[None, :, :]</code>                        |
| Batch matrix product by scalar coefficient | <code>c[:, None, None] * M[None, :, :]</code>     |
| Batched Hermitian conjugate                | <code>H.swapaxes(-1, -2).conj()</code>            |
| Batched Hermitian eigenvalues              | <code>np.linalg.eigvalsh(H_batch)</code>          |
| Typical Hamiltonian batch shape            | <code>(N_points, N_orb, N_orb)</code>             |
| Typical eigenvalue shape                   | <code>(N_points, N_orb)</code>                    |
| Chemical-potential shift                   | <code>energies = eigvalsh(H) - mu</code>          |
| Chunk if memory is large                   | <code>for start in range(0, N, chunk): ...</code> |

### Final idea

Batching is mainly a change in how you organize the data. Once every physical parameter is assigned its own array axis, NumPy broadcasting and batched linear algebra can replace many explicit Python loops.

For numerical physics, always think in terms of what each axis means physically. That makes high-dimensional array code far easier to understand and debug.